



SENSOR ATMOSFERYCZNY Z BLUETOOTH

dokumentacja

Autor: Eryk Wicher

Spis treści

Lista oznaczeń	2
1. Wstęp	2
2. Analiza problemu	3
2.1. Bluetooth (BLE)	3
2.2. Połączenie mikrokontroler – czujnik	4
2.3. Aplikacja w Androidzie	5
3. Implementacja	7
3.1. Komunikacja mikrokontroler-aplikacja:	7
3.2. Aplikacja w Androidzie:	10
3.3. Wygląd sensora	13
4. Podręcznik użytkownika	14
5. Opis wykonanych testów i usterki	16
6. Użyty sprzęt oraz oprogramowanie	17
7. Bibliografia	17

Lista oznaczeń

BLE – bluetooth low Energy

UI – user interface

1. Wstęp

Dokument ten dotyczy sensora działającego na płytce arduino uno r4 wifi wykonującego pomiary temperatury oraz wilgotności oraz przesyłający te dane za pomocą BLE na aplikację na androidzie.

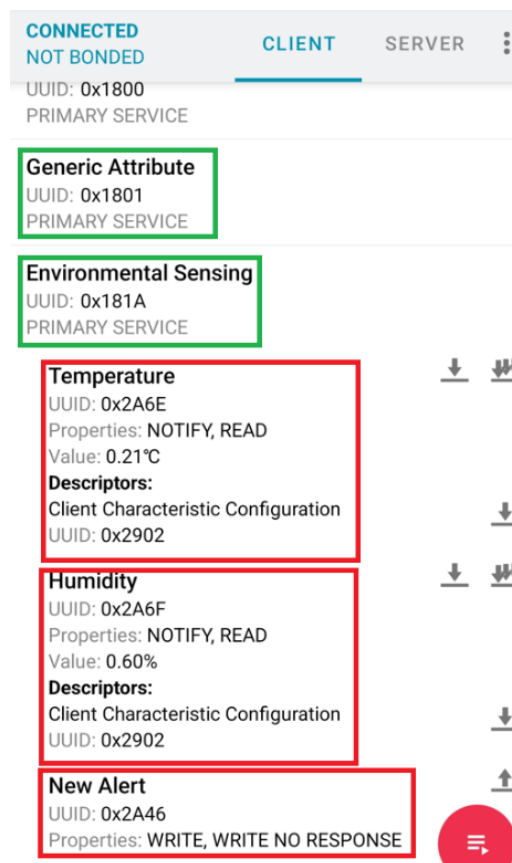
2. Analiza problemu

2.1. Bluetooth (BLE)

BLE to energooszczędna odmiana standardu bluetooth. Ten standard posiada wiele udogodnień dla użytkownika i projektanta. Głównymi elementami są usługi i charakterystyki.

Te pierwsze to predefiniowane lub niestandardowe mówi czym zajmuje się dane urządzenie (np. odczyty środowiskowe, aktualny stan baterii czy informację o urządzeniu). Jednocześnie może być uruchomiona więcej niż jedna usługa.

W obrębie każdej z usług działają charakterystyki, które także mogą być predefiniowane lub niestandardowe oraz może być ich wiele. Są one konkretnym typem danych np. temperatura czy prędkość.



Zrzut ekranu z programu nrf connect służącego do skanowania urządzeń bluetooth. Na zielono zostały zaznaczone usługi, a na czerwono charakterystyki.

W tym ten sensor realizuje usługę odczytów środowiskowych (Environmental Sensing Service), w której znajdują się trzy charakterystyki:

- „temperatura”
- „wilgotność”
- „nowe powiadomienie” → zostało użyte jako metoda wymuszenia wykonania pomiaru

Charakterystyki mogą mieć różne właściwości takie jak:

- „odczyt”
- „zapis”
- „powiadomienia” → możliwość za subskrybowania do danej charakterystyki

Aby w BLE pobrać dane należy o nie prosić za każdym razem lub za subskrybować je. Wtedy sensor automatycznie wyśle daną wartość jeżeli tylko zostanie ona zaktualizowana.

2.2. Połączenie mikrokontroler – czujnik

Czujnik jest wpięty bezpośrednio do wyprowadzeń mikroprocesora oraz korzysta z autorskiego systemu komunikacji dostarczonego przez producenta. Posiada on sumy kontrolne oraz działa tylko na jednej linii.

2.3. Aplikacja w Androidzie

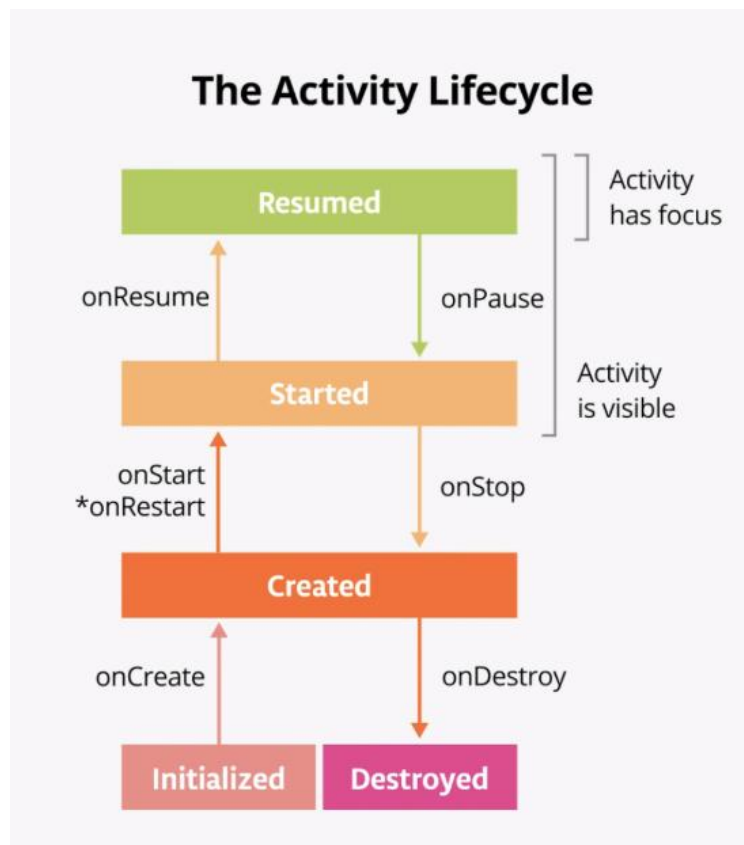
Aplikacja została napisana w Android Studio za pomocą języka Kotlin i frameworku Jetpack Compose. Zapewni on proste klasy do budowania UI.

Samo budowanie UI opiera się na tworzeniu kontenerów np. kolumn lub wierszy (które są klasami) i wkładanie do nich innych kontenerów lub przycisków czy pól tekstowych.

Główną klasą tutaj jest MainActivity, które dziedziczy po ComponentActivity. Można to porównać do funkcji main, która jest najczęściej używana w innych językach programowania.

Jest to klasa zapewniana przez framework, która jest sercem aplikacji w Androidzie. To właśnie w niej jest metoda `.onCreate()`, w której jest rysowane całe UI. Zawiera też inne metody, które określają na przykład co się stanie gdy aplikacja zostanie zminimalizowana lub zamknięta itd.

Cały framework opiera się na poniższych stanach:



Źródło - <https://developer.android.com/courses/pathways/android-basics-compose-unit-4-pathway-1>

Programista nie ma wpływu na to w jakim stanie obecnie znajduje się aplikacja i nie może tego w żaden sposób wymusić. Może jedynie określać co się dzieje gdy zmieniane są stany poprzez nadpisywanie metod `onCreate`, `onStart` itd.

Aktualizowanie UI w tym frameworku jest wykonywane automatycznie dzięki specjalnym zmiennym stworzonym za pomocą funkcji `mutableStateOf()`. Taka zmienna jest obserwowana przez framework i gdy zmieni wartość, a ona znajduje się gdzieś na UI to ten zostanie zaktualizowany.

3. Implementacja

3.1. Komunikacja mikrokontroler-aplikacja:

System używa BLE z usługą odczytów środowiskowych (Environmental Sensing Service), w której zawierają się poniższe charakterystyki:

- „temperatura”
- „wilgotność”
- „nowe powiadomienie” → zostało użyte jako metoda wymuszenia wykonania pomiaru

W przypadku mikroprocesora producent udostępnił bardzo wysoko poziomą bibliotekę umożliwiającą tworzenie aplikacji BLE o nazwie „ArduinoBLE.h”. Zawiera ona obiekt „BLE”, który jest odpowiedzialny za całą komunikację. Do powyższego obiektu dodaje się usługi, a do nich charakterystyki.

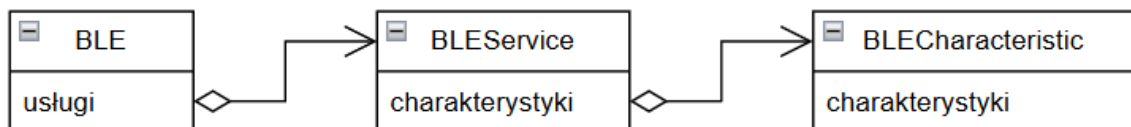


Diagram klas dla BLE na mikroprocesorze.

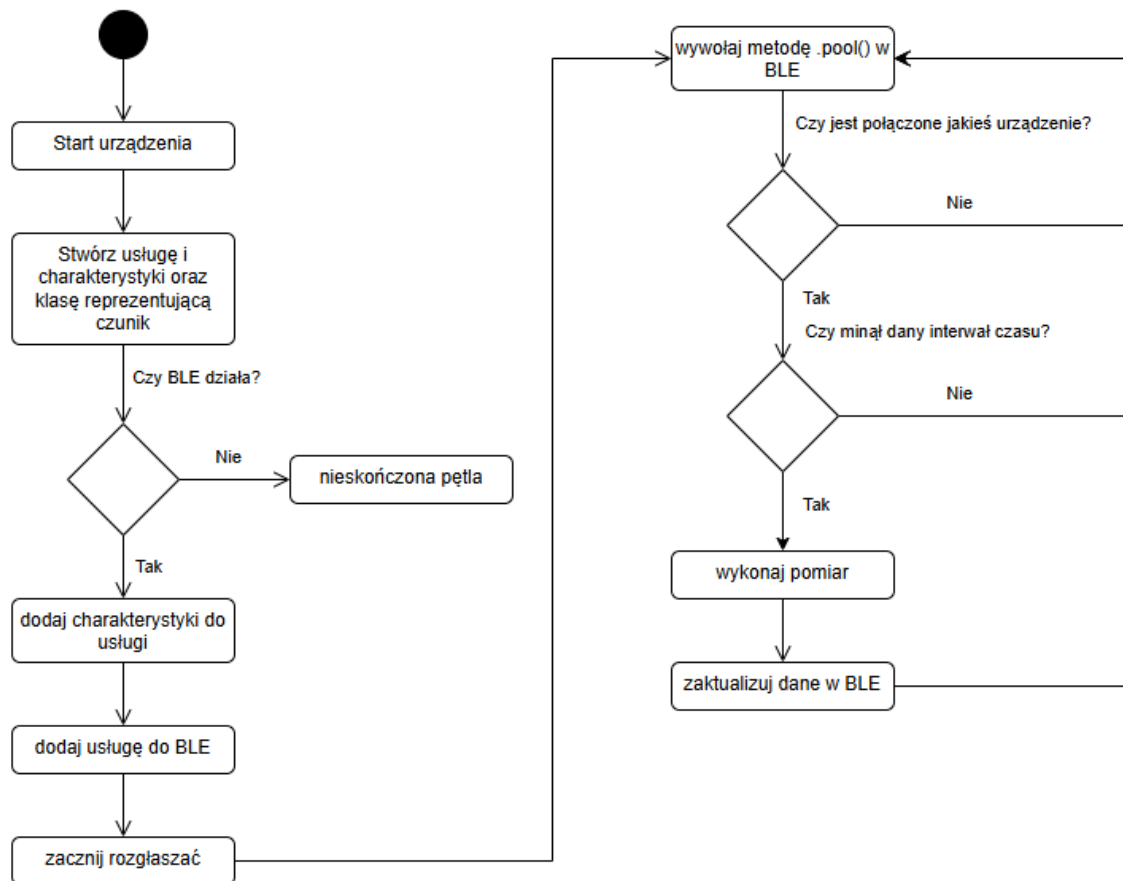


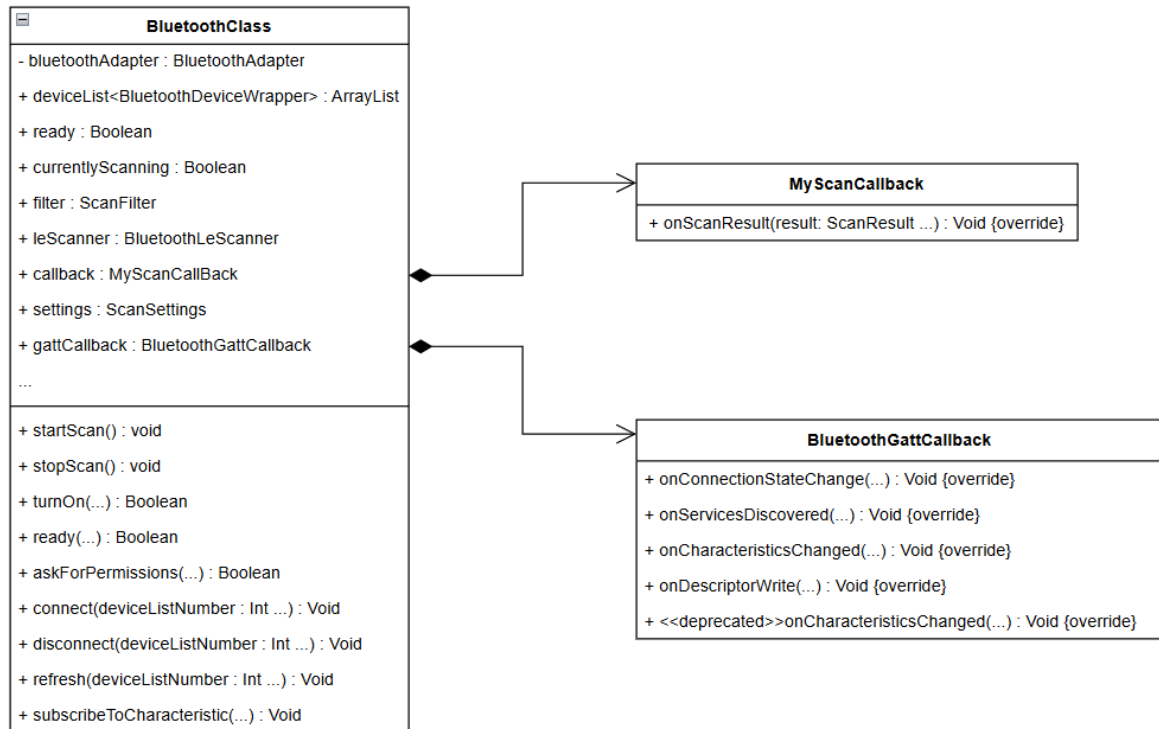
Diagram przepływu programu na mikrokontrolerze.

Natomiast konfiguracja BLE w aplikacji na system android wygląda zupełnie inaczej, ponieważ w tym przypadku BLE nie jest zarządzane przez programistę tylko przez system. Dlatego też komunikacja wygląda tu zupełnie inaczej.

Na początku należy sprawdzić w systemie czy dana aplikacja ma pozwolenie od użytkownika na używanie BLE. Jeżeli nie ma to trzeba poprosić przez system o przyznanie ich. Następnie trzeba sprawdzić czy adapter BLE jest włączony. Jeżeli nie to także trzeba poprosić system.

W tym momencie aplikacja ma pełny dostęp do BLE.

Cała biblioteka zapewniona przez Andoida opiera się na funkcjach zwrotnych (callback). Na szczęście klasy abstrakcyjne są już zapewnione więc wystarczy nadpisać odpowiednie metody.



Klasa odpowiedzialna za BLE. Klasy **MyScanCallback** i **BluetoothGattCallback** to nadpisane klasy abstrakcyjne.

Nadpisane klasy:

- **MyScanCallback** → podczas skanowania jest ona wywoływana gdy system android znajdzie inne urządzenie BLE. Jest tak nadpisana aby zapisywała wyniki tych skanów do listy urządzeń znajdujących się w **BluetoothClass**.
- **BluetoothGattCallback** → jest ona podawana jako argument funkcji do łączenie się z innym urządzeniem BLE. Zawiera metody, które np. definiują co się stanie gdy stan połączenie się zmieni lub gdy sensor wyśle dane.

3.2. Aplikacja w Androidzie:

Jak już wspomniano powyżej aplikacja została napisana w Android Studio z użyciem frameworka Jetpack Compose oraz składa się z 2 klas:

- MainActivity
- MainComposition

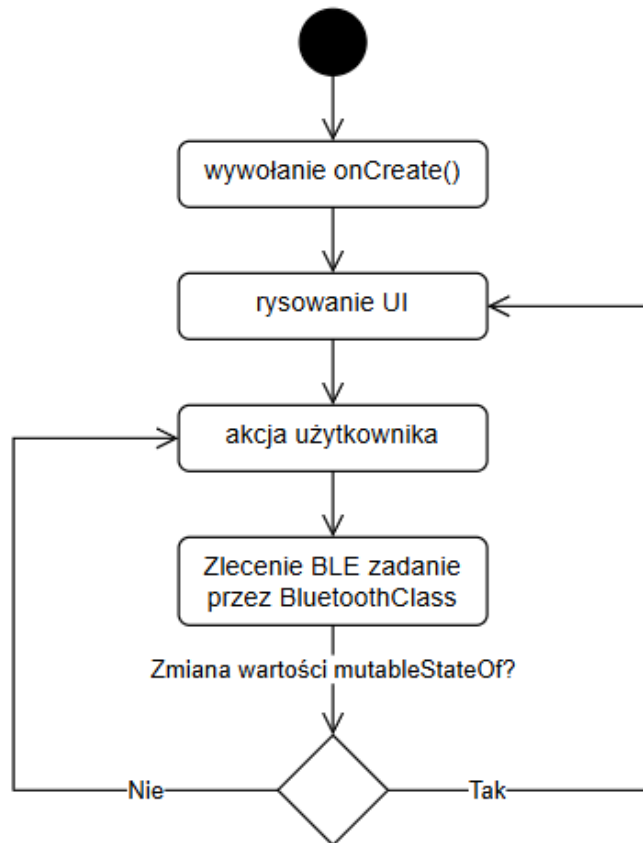
MainActivity
+ bluetooth : BluetoothClass
+ onCreate(...) : Void {override}
+ onStop(...) : Void {override}
+ onStart(...) : Void {override}

MainComposition
+ Comp(blueetoothClass : BluetoothClass, ...) : Void

Klasy aplikacji.

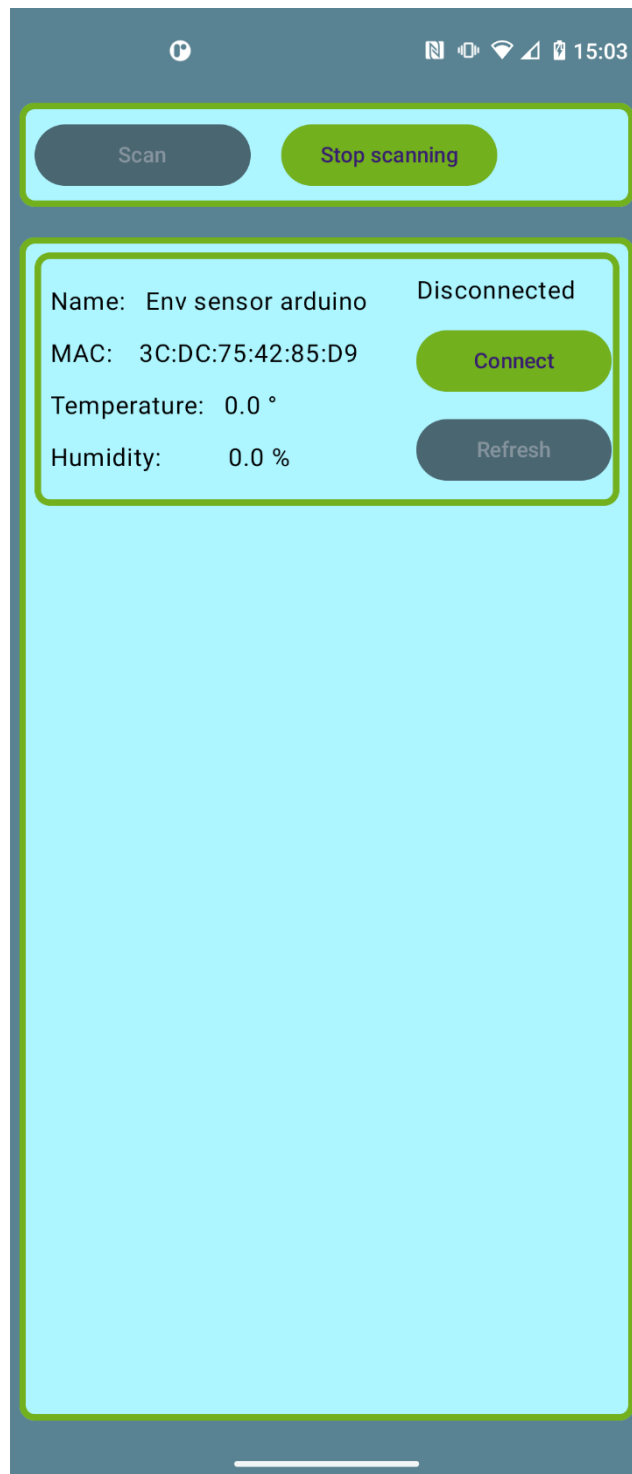
Klasa MainActivity zawiera obiekt klasy BluetoothClass, który został przedstawiony powyżej oraz który zapewnia całą obsługę BLE udostępniając tylko metody do sterowania.

Klasa MainComposition ma tylko jedną statyczną metodę Comp(), w której są instrukcje dotyczące wyglądu interfejsu. Jest ona wywoływana w metodzie onCreate(). Ponieważ w UI jest wiele przycisków, które sterują BLE to należało przekazać obiekt bluetooth z MainActivity.



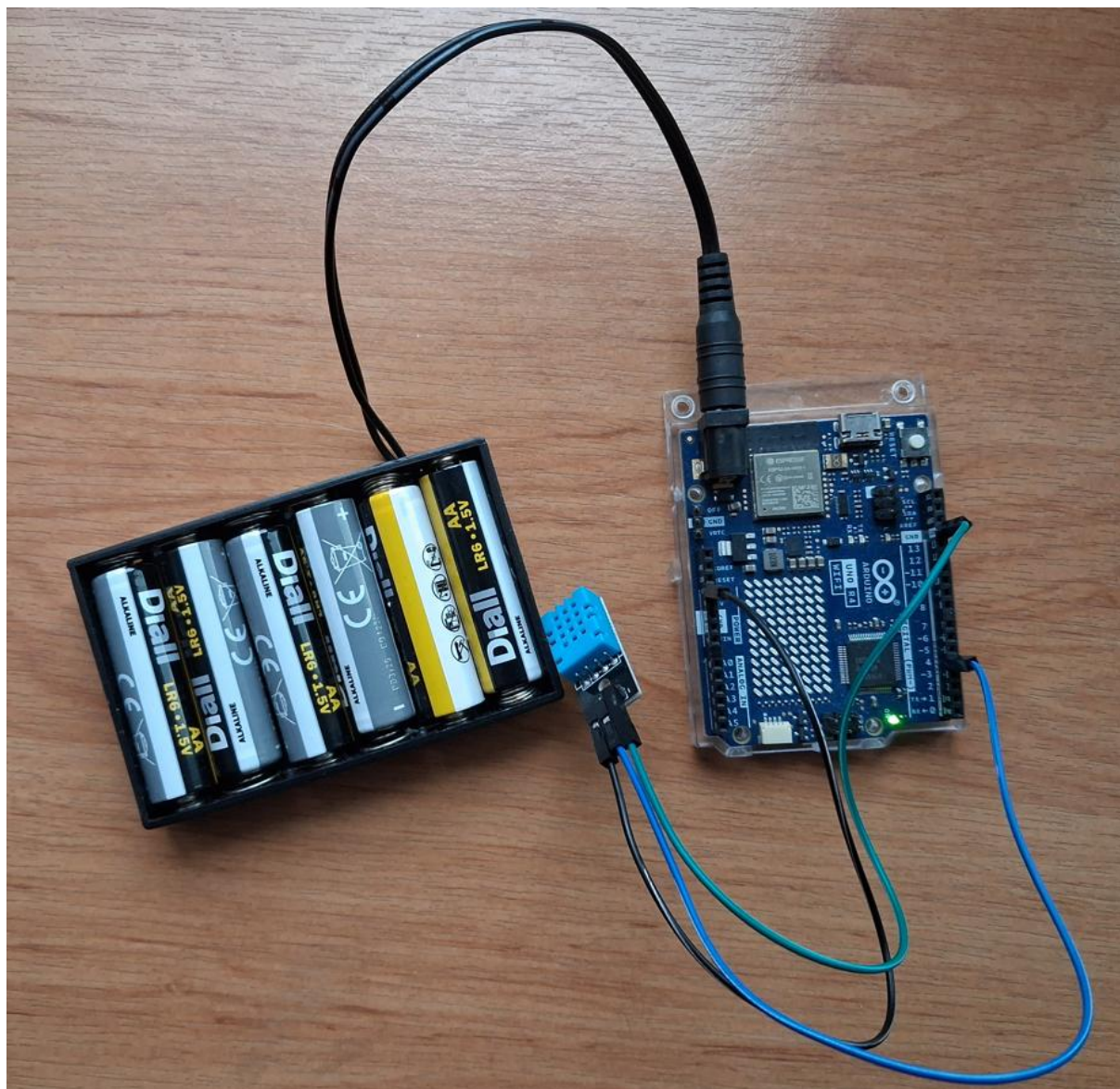
Uproszczony diagram przepływu aplikacji na Adnroida.

Na powyższym diagramie akcja użytkownika to np. naciśnięcie jednego z przycisków. Wywołują one metody BluetoothClass, a te z kolei zlecają systemowi wykonanie jakiejś akcji BLE. Wynik tej akcji odbieramy funkcjami zwrotnymi (callback), które to najczęściej zmieniają wartości zmiennych mutableStateOf dzięki czemu UI jest aktualizowane.



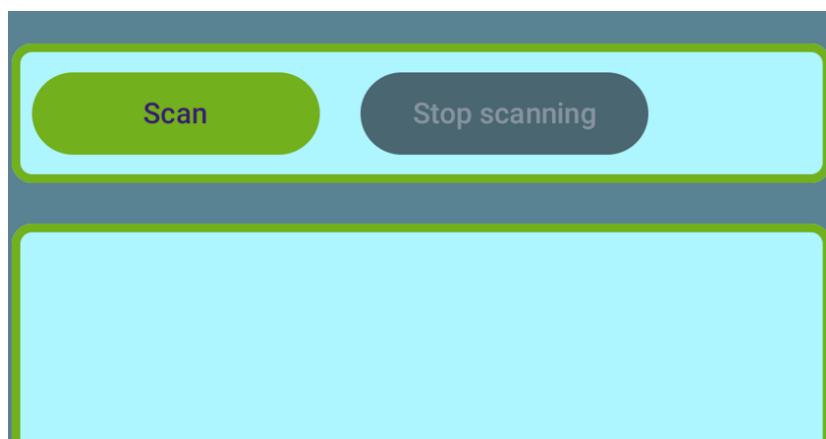
Wygląd aplikacji.

3.3. Wygląd sensora



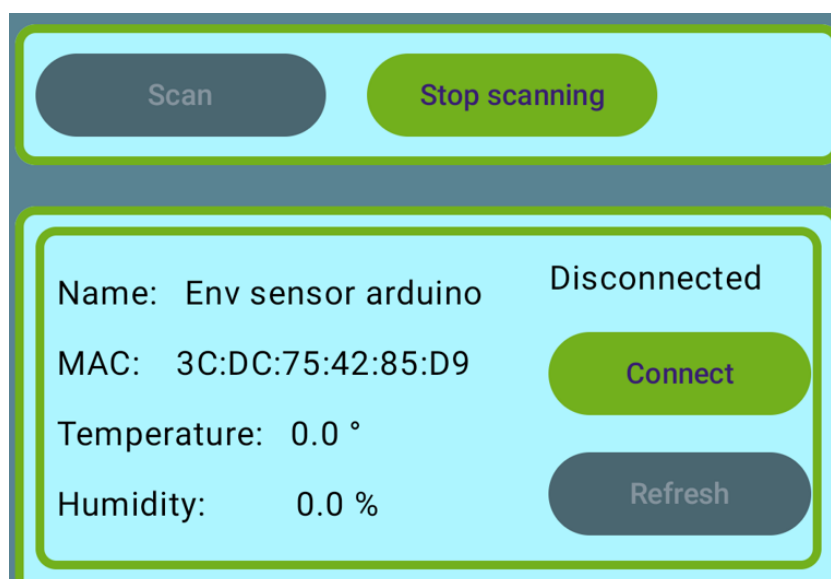
4. Podręcznik użytkownika

Po włączeniu aplikacji wyświetli się poniższy widok.



Z czego tylko przycisk Scan może zostać naciśnięty. Jeżeli aplikacja nie ma wymaganych uprawnień lub adapter BLE jest wyłączony w systemie pierwsze naciśnięcie tego przycisku spowoduje wyświetlenie się systemowych okienek z prośbą o przyznanie uprawnień i włączenie adaptera.

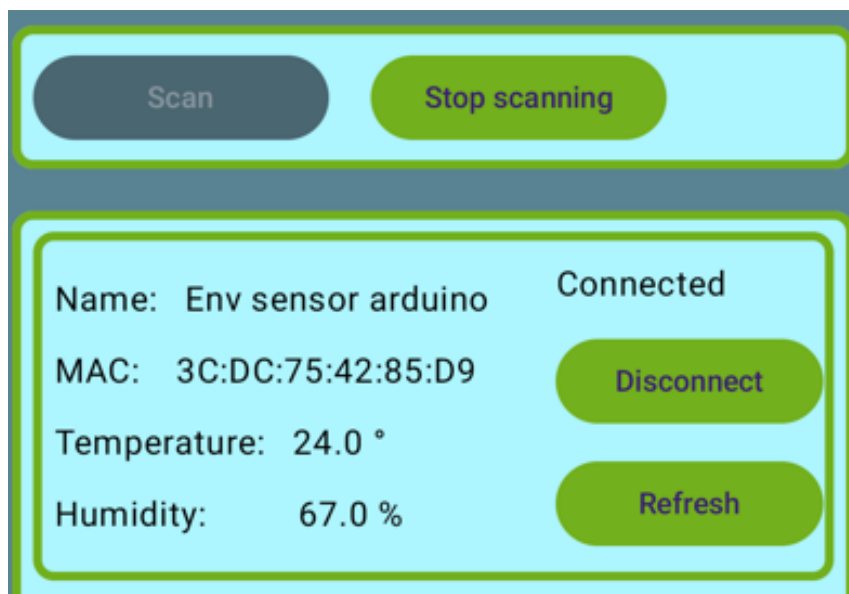
Kolejne naciśnięcie tego przycisku spowoduje rozpoczęcie skanowania, a wyniki zostaną wyświetlone w ramce poniżej. Skanowanie można i nawet należy zatrzymać jeżeli połączenie z sensorem zostało już zrealizowane gdyż pozwoli to na oszczędzanie baterii.



Aby się połączyć z sensorem należy nacisnąć przycisk Connect obok odpowiedniego urządzenia. W ramce powinny pokazywać się tylko kompatybilne urządzenia gdyż są one przepuszczane przez filtr. Jednak jeżeli inne urządzenie posiada tę samą usługę (odczyty środowiskowe) to może się zdarzyć, że zostanie ono wyświetlone.

Jednakże nie będzie można się z nim połączyć gdyż na tym etapie jest sprawdzane czy urządzenie posiada dokładnie te charakterystyki, które są oczekiwane.

Po poprawnym połączeniu wygląd zmieni się na poniższy, a po chwili zmienią się również wartości odczytywane z czujnika.



Wartości temperatury i wilgotności będą automatycznie aktualizowane co określony interwał czasu jednak można wymusić pomiar przyciskiem refresh.

5. Opis wykonanych testów i usterki

Pierwszą napisaną aplikacją była ta na mikroprocesor oraz była testowana za pomocą monitora szeregowego, który wypisywał wiadomości na konsolę na komputerze. Urządzeniem odbierającym dane był telefon z androidem z aplikacją nRF Connect, która to pozwala na sprawdzanie praktycznie każdej funkcjonalności BLE. Ponieważ na tym etapie aplikacja była dość prosta więc nie znaleziono błędów wartych uwagi.

Następnie napisano aplikację na Androida. Tutaj testy były przeprowadzane na dwóch telefonach: Samsung Galaxy M13 z Androidem 14 oraz Motorola moto g 5G plus z Androidem 11. Ważne było aby testować na dwóch różnych wersjach Androida gdyż w 12 i 13 znacząco zmieniono działanie BLE w systemie. Testowanie było wykonywane ręcznie, a patrzono na to czy system działa czy nie.

Znalezione błędy:

- BLE działa na androidzie 14, a na 11 już nie – rozwiązaniem było użycie funkcji oznaczonej jako przestarzała (deprecated) gdyż pomimo tego przypisu była ona przeznaczona na starsze wersję Androida np. 11.
- Klasa BluetoothClass była niszczona przy obracaniu ekranu (w takim przypadku system niszczy i tworzy całą aplikację na nowo) – ponieważ ta klasa posiada listę wykrytych urządzeń jak i wszystkie połączenia BLE to musi przetrwać obrót ekranu. Rozwiązaniem było zainicjalizowanie obiektu klasy BluetoothClass jako ViewModel.

6. Użyty sprzęt oraz oprogramowanie

Telefony do testów:

- Samsung Galaxy M13 Android 14
- Motorola moto g 5G plus Android 11

Mikrokontroler:

- Arduino UNO R4 WiFi

Czujnik atmosferyczny (pomiary temperatury i wilgotności):

- DTH11 - <https://botland.com.pl/czujniki-multifunkcyjne/1886-czujnik-temperatury-i-wilgotnosci-dht11-modul-przewody-5903351242448.html>

IDE do Arduino:

- Arduin IDE 2.3.6

IDE do aplikacji Androida:

- Android Studio Panda 2 | 2025.3.2

Aplikacja do testowania BLE:

- nRF Connect

7. Bibliografia

- Kurs frameworku Compose - <https://developer.android.com/courses/android-basics-compose/course>
- Dokumentacja androida i Compose
- Przykłady i dokumentacja ArduinoBLE
- Diagramy narysowane w Draw.io - <https://app.diagrams.net/>